FIELD REPORT 01 / CWE-78 / IN-BAND

OS Command Injection

Exploitation and mitigation in a local checksum application

1. Project brief

Checksum Lab demonstrates how a filename can become an operating-system instruction when a web server concatenates user input into a shell command. Two implementations receive identical input: one deliberately vulnerable, and one secured with a native hashing API.

Item	Detail
Student / roll number	Manas Singh / IIB2024014
Institute	Indian Institute of Information Technology Allahabad
Course / semester	Cyber Security (IT-30001) / Autumn 2026
Instructor	Dr. Soumyadev Maity
Assigned category	OS command injection
Selected variant	Variant 1: in-band, with command output returned
Primary mitigation	Avoid shell execution: use Python hashlib
Prepared	27 October 2026

Objective and scope

Implement a vulnerable path, show a working exploit, explain its root cause, implement a mitigation, and show the same payload failing after the fix. Explain the other command injection mitigations named in the assignment. This project supplies one member's category; other group members must contribute their own distinct categories as applicable.

Architecture

Browser input -> JSON POST -> Python HTTP handler -> vulnerable or secure checksum function -> JSON response -> side-by-side browser output.

The vulnerable branch invokes `/bin/sh` through `subprocess.Popen(..., shell=True)` and returns captured command output. The secured branch checks a fixed filename allow-list and computes the digest in-process using `hashlib.sha256`. All three sample files are synthetic.

Start the demonstration

Docker Engine/Desktop and the Compose plugin are required. No pip packages or frontend build tools are needed.

```
docker compose up --build -d --wait
# Open http://127.0.0.1:8080
docker compose down
```

Run the lab only in its provided local container. Its vulnerable route intentionally accepts shell syntax. The server's normal startup refuses to run outside Docker or as root.

2. Vulnerable implementation and exploit

(a) Identify the flawed code path

`POST /api/vulnerable` -> `Handler.do_POST()` -> `run_checksum()` -> `vulnerable_checksum()` in `app/server.py`. The following is an excerpt; response handling is omitted.

```
command = f"{SHA256SUM} -- {filename}"
process = subprocess.Popen(
    command,
    shell=True,              # intentional flaw
    cwd=SAMPLES,
    stdin=subprocess.DEVNULL,
    stdout=subprocess.PIPE,
    stderr=subprocess.PIPE,
    text=True,
    # other process options omitted
)
stdout, stderr = process.communicate(timeout=2)
```

(b) Exact working exploit

The normal value is `welcome.txt`. Replace it with this exact value:

```
welcome.txt; printf 'OS_INJECTION_CONFIRMED\n'
```

The command string becomes:

```
/usr/bin/sha256sum -- welcome.txt; printf 'OS_INJECTION_CONFIRMED\n'
```

The shell runs the checksum utility, then executes `printf`. The marker is harmless and requires no secrets, file changes, or outbound request. The output is returned in the same HTTP response: this is an in-band exploit.

Exact HTTP request

```
POST /api/vulnerable HTTP/1.1
Host: 127.0.0.1:8080
Content-Type: application/json
X-Lab-Request: 1

{"filename":"welcome.txt; printf 'OS_INJECTION_CONFIRMED\\n'"}
```

The double backslash in JSON encodes one literal backslash in the filename string. The included `evidence/payload.json` can be sent directly with curl; the HTTP client computes Content-Length.

(c) Root cause

Untrusted input is concatenated into a string evaluated by a shell. The shell interprets `;` as a command separator, allowing input intended as a filename to introduce a second command. The utility's `--` ends option parsing; it does not disable shell grammar.

3. Implemented mitigation and alternatives

(d) Fixed code path

`POST /api/secure` -> `run_checksum()` -> `secure_checksum()` in `app/server.py`. The following excerpt shortens the response dictionaries only.

```
ALLOWED_FILES = (
    "welcome.txt", "release-notes.txt", "invoices.csv"
)

def secure_checksum(filename):
    if filename not in ALLOWED_FILES:
        return {"status": "blocked", "stdout": "", ...}
    digest = hashlib.sha256(
        (SAMPLES / filename).read_bytes()
    ).hexdigest()
    return {"status": "accepted", ...}
```

The primary fix removes the OS command and shell entirely. The checksum is calculated by a Python library, leaving no command interpreter to evaluate separators or substitutions. An exact server-side allow-list is an additional implemented control that restricts access to the intended samples and rejects traversal paths.

The identical exploit input is rejected with HTTP 422 and an empty `stdout`. Tests also replace `subprocess.Popen` with a failure stub and verify that the secured path never invokes it for either valid input or the exploit. This distinguishes removing shell execution from merely blocking one example string.

(e) Other mitigation ideas

Argument-vector execution. If an external utility is necessary, pass a fixed executable and separate arguments with `shell=False`. For example:

```
subprocess.run(
    ["/usr/bin/sha256sum", "--", validated_filename],
    shell=False, capture_output=True, text=True, timeout=2
)
```

Shell metacharacters then remain argument data. Continue validating paths and preventing option or executable injection; avoiding a shell does not make every possible utility invocation safe.

Allow-list validation. Accept only expected values on the server. A fixed set is suitable for this tiny sample catalog. Broad user-controlled paths require a more careful file-access design. Client-side checks can be bypassed; rejecting a few special characters is an incomplete blacklist.

Least privilege. Run under a restricted account with limited filesystem and network access. This reduces the consequences of exploitation but does not repair unsafe command construction. The project's non-root/read-only container configuration is a supporting control.

Avoiding OS commands. Prefer native APIs when they provide the required functionality. This is the mitigation actually selected here. Shell-specific quoting is more error-prone and can leave argument-level abuse; it is not this project's selected fix.

These explanations follow the categories in the assignment. Compare them with the lecture slides before the viva, because the slides were not supplied.

4. Observed exploit and blocked attempt

The browser screenshot below shows the same payload sent to both implementations. The left response contains the injected marker; the right response rejects the input before any child process is created.

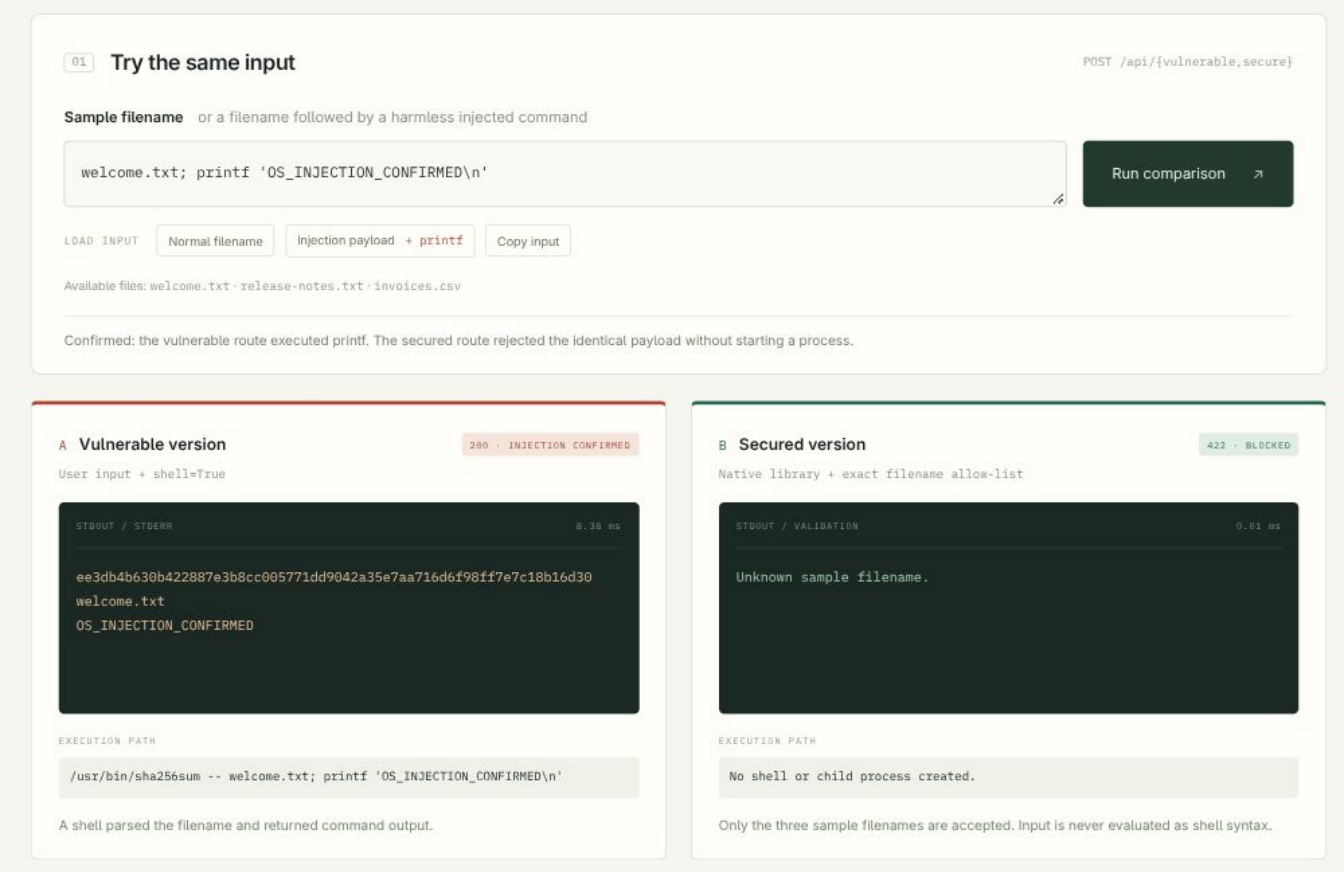


Figure 1. Actual local browser evidence: successful in-band execution and the identical blocked attempt. Full baseline and exploit screenshots are included in `evidence/`.

Captured request results

Input	Vulnerable response	Secure response
welcome.txt	HTTP 200, checksum returned	HTTP 200, identical checksum
Exact injection payload	HTTP 200, marker in stdout	HTTP 422, blocked, stdout empty

Both baseline implementations returned the SHA-256 digest:

```
ee3db4b630b422887e3b8cc005771dd9042a35e7aa716d6f98ff7e7c18b16d30
```

The vulnerable response adds a separate line containing `OS_INJECTION_CONFIRMED`. The fixed response contains `Unknown sample filename.` in the error field and returns no command output. The browser renders all output as text using `textContent`.

The four raw requests, response bodies, timestamps, and source hashes are in `evidence/requests.json`; a readable transcript is in `evidence/transcript.txt`. Evidence was captured on 27 October 2026. Execution timings vary between runs and are not the proof of this in-band variant.

5. Validation and operational limits

Reproduce the evidence

Start the lab, then run the host-side verifier using Python 3.10 or later:

```
python3 scripts/verify.py
```

All seven HTTP checks passed: both baseline status codes, matching baseline digests, successful exploit status, returned marker, identical attack body, blocked status, and absent secured output. The script checks only this lab's localhost address.

Regression suite

```
docker compose run --rm --no-deps lab \
  python -m unittest discover -s tests -v
```

Observed result: 13 tests passed. The suite verifies behavior, real exploitation, and the security boundary rather than checking only a displayed success message.

Test area	What is checked
Baseline	All three sample files return the same digest in both versions
Exploit	A second shell command prints the marker
Fix	Identical payload blocked; no process for valid or invalid secure inputs
Input boundaries	Other separators/substitution, invalid paths, types, lengths, null byte
Command lifetime	A harmless long-running command is terminated after two seconds
HTTP behavior	Real exploit and secured responses; bad JSON shape rejected
Local browser protection	Cross-origin requests, missing custom header, and rebinding hosts rejected
Static resources	Explicit routes, security headers, traversal request rejected

The saved test log is `evidence/tests.txt`. A GitHub Actions workflow builds and tests the same image with networking disabled in the test container; it has not yet run on GitHub because no repository has been published.

Container configuration verified locally

The running process used Python 3.12.16 and UID 10001. The sample directory was not writable. Inspection confirmed a read-only root filesystem, all capabilities dropped, no-new-privileges, 128 MiB memory limit, 48-process limit, and only `127.0.0.1:8080` published to the host. No host bind mounts are configured.

The normal lab uses Docker's ordinary bridge network and can make outbound connections. It is not an offline sandbox or a complete security boundary. Use only the harmless supplied payloads, keep the loopback binding, and stop the container after the demonstration. The output is truncated after capture; resource and time limits reduce risk but are not a general defense against malicious workloads.

This demonstrates and fixes a specific command injection path. It is not a production web server or a claim that the entire application has been security-audited.

6. Narrated demonstration: 2-3 minutes

Use this as a rehearsal outline in your own words. Record your own screen and voice; screenshots and this script do not satisfy the required video by themselves. Explain only work you have reviewed, adapted as needed, and can defend under the course's originality/AI-assistance rules.

0:00-0:20 - Introduce ownership and variant

Say: "I am Manas Singh, roll number IIB2024014. My category is OS command injection, and I am demonstrating the in-band variant. This is a local sample-file checksum application. The command output comes back in the same HTTP response."

Show the application title, your identity, and the two implementations. Mention that all testing is against your own local lab.

0:20-0:40 - Establish normal behavior

Keep `welcome.txt` in the input and click **Run comparison**. Point to the matching SHA-256 digests and HTTP 200 responses. Explain that both versions perform the same intended task on valid input.

0:40-1:10 - Demonstrate the exploit

Click **Injection payload**, read the exact string, and click **Run comparison**. Point to `OS_INJECTION_CONFIRMED` in the vulnerable output.

Say: "The semicolon ends the checksum command and begins another shell command. Here the second command is a harmless printf. The server really executes it and returns the marker."

1:10-1:35 - Explain the root cause in source

Open `app/server.py`, find `vulnerable_checksum()`, and point to the formatted command string and `shell=True`.

Say: "The filename is untrusted input. Concatenating it into a string evaluated by a shell lets data become syntax. The utility's double hyphen protects option parsing, but it does not protect against a shell separator."

1:35-2:05 - Explain and show the mitigation

Keep the exact same payload visible. Point to HTTP 422 and the blocked result on the secured side. Show `secure_checksum()` in the source.

Say: "The primary fix uses hashlib directly, so there is no operating-system command or shell. I also restrict the filename to the three sample files. The same attack is rejected, and no child process is started. Valid filenames still work."

2:05-2:40 - Cover alternatives and verification

Say: "If an external process were necessary, I would pass a fixed executable and separate arguments with shell=False and validate the arguments. A narrow server-side allow-list blocks unexpected values. Least privilege reduces the damage if a vulnerability remains. It does not repair unsafe shell construction."

Show the passing test log briefly. State that the test suite checks both the working exploit and the absence of process creation in the fixed path. End the recording with the successful exploit and blocked result visible.

Recording checklist

- Keep the payload, output, and status codes readable; use browser zoom if needed.
- Show the actual source functions as well as the UI.
- Use the same payload for the before/after comparison.
- Narrate your own segment; combine it with teammates' segments when applicable.
- Verify the submitted video has audible voice-over and a working link.

7. Viva questions and concise answers

What is OS command injection? User-controlled data changes the structure or behavior of an OS command the application executes. In this example, a filename introduces a second shell command.

Why is this in-band rather than blind? The injected command's output is captured and returned in the HTTP response. A blind variant would infer execution from timing or another side effect instead of reading the output here.

Which exact lines cause it? The formatted string includes the user-controlled filename, and `subprocess.Popen(..., shell=True)` sends that string to a shell. The `VULNERABLE SINK` comment marks the path in `app/server.py`.

Why use printf rather than a destructive command? Printing a fixed marker is enough to prove unintended command execution. It avoids accessing private data, modifying files, or contacting another host.

Does -- prevent this payload? No. It ends the checksum utility's option parsing. The shell still interprets the semicolon before launching the commands.

What exactly is the implemented mitigation? Replace shell-based checksum execution with `hashlib.sha256`. The route also implements an exact filename allow-list to keep file access within the sample dataset.

Is a blocked screenshot sufficient to establish that the shell was removed? It shows the required example failing. Source inspection and tests that fail if `Popen` is invoked provide the additional evidence that the secured path does not start a process.

Would shell=False alone solve every command-related problem? It prevents shell interpretation when correctly using a fixed executable and argument list. It does not automatically prevent dangerous utility options, arbitrary executable selection, or unauthorised file access.

Why not blacklist semicolons? Shells have other operators and substitution syntax. A narrow allow-list is easier to reason about, and removing the shell eliminates this interpreter boundary.

Why run as a non-root user? Injected commands inherit the process's permissions. Reducing those permissions limits impact, but does not remove the flaw itself.

Why return HTTP 422? The JSON request is readable, but the filename is not an accepted sample identifier. The status is a design choice for validation feedback; the actual defense is in the server code, not the numeric status.

What are the limits of this demonstration? It covers one in-band Linux-shell checksum path. The normal container has outbound bridge-network access, and the deliberately vulnerable route remains present for teaching. Do not publish the running app publicly.

Primary references

- OWASP OS Command Injection Defense Cheat Sheet - defense categories and argument injection considerations.
- Python subprocess security considerations - how explicit shell invocation changes interpretation and quoting responsibility.
- MITRE CWE-78 - weakness definition and consequences.

Source assignment: `Assignment_Web_Server_Security.docx.pdf`, supplied by the student, pages 1-2. Primary web references checked on 27 October 2026. The lecture slides and course submission page were not supplied.

Related: OS Injection Lab - Technical Report, OS Injection Lab - Evidence and Submission.